

mdstats Universal 3-D Graphics Architecture Standard

Normative architecture for composable scientific visualization layers, shared scene dependencies, renderer-neutral provenance, and a configurable 3-D graphics CLI

mdstats architecture manual

2026-08-11

Contents

Document status	4
Source consolidation and authority	5
Architectural objective	7
Design principles	7
Scientific ownership before graphics convenience	7
Independent layers, shared dependencies	8
Scientific preparation is not rendering	8
Render options are not scientific options	8
One coordinate gauge per scene	8
Display replication is not scientific duplication	8
Arbitrary combinations must be first-class	8
Configuration must compile to one canonical scene declaration	8
Fail closed on ambiguity	8
Visibility is not execution authority	8
Resource management is scene-wide	9
Current baseline and architectural pressure	9
Current composite scene	9
Current composite render result	9
Current prototype CLI	9
Dependency direction and ownership	10
Terminology	11
Universal scene contract	11
Target object	11
Scene invariants	11
Layer order	12
Universal layer contract	12
Layer instance identity	12
Multiple instances	12
Required interface	12
Required versus optional dependencies	12
Universal selection model	13
Purpose	13
Core selection vocabulary	13
Selection semantics	13
Stable identifiers	13
Ambiguity policy	13
Scene context and dependency DAG	14
Shared scene context	14
Dependency keys	14
DAG construction	14
Parallel dependency execution	15

Cache scope	15
Scientific, render, and execution identity	15
Scientific identity	15
Render identity	15
Execution identity	16
Canonical scene manifest	16
Purpose	16
Manifest content	16
Manifest serialization	17
Layer registry and extension model	17
Registry requirement	17
Internal-first plugin policy	17
Registry metadata	17
Initial layer family: framework topology	17
Scientific ownership	17
Layer purpose	17
Dependencies	18
Render controls	18
Initial layer family: atomic connectivity	18
Scientific distinction	18
Layer modes	18
Dependencies	18
Pair selection	18
Initial layer family: atomic trajectory	18
Scientific product	18
Selection	18
Path modes	19
Registration	19
Render options	19
Initial layer family: atomic density	19
Scientific ownership	19
Layer request	19
Multiple densities	19
Resource integration	19
Framework density and other current density products	19
Future layer families	20
Ring layer	20
Cage layer	20
Site layer	20
Site-assignment layer	20
Transition-path layer	20
Markov/kinetics layer	20
Free-energy and vector layers	21
Coordinate system, registration, and display gauge	21
Scene-level authority	21
Layer compatibility	21
Display cell	21
Camera/view transform	21
Periodic display and replication	21
Renderer-neutral graphical primitives	21
Motivation	21
Initial primitive set	21

Primitive provenance	22
Renderer role	22
Renderer interface	22
Generic render result	22
Legend, visibility, and interaction model	23
Generic layer tree	23
Initial visibility	23
Layer opacity	23
Legend identity	23
View presets and reproducible snapshots	23
CLI architecture	24
Thin launcher	24
CLI role	24
Source formats	24
Declarative configuration	24
TOML as the complex-scene format	24
CLI shorthand	25
Precedence	26
Preset architecture	26
Presets are configuration	26
Legacy preset	26
Resource architecture	26
One authority	26
Cross-layer admission	26
Browser resources	27
Progress and diagnostics	27
Failure semantics	27
Output architecture	27
Initial output	27
Sidecar evidence	27
Future outputs	28
Compatibility and migration	28
FrameworkDynamicsScene	28
FrameworkDynamicsRenderResult	28
plot_framework_dynamics_3d	28
Hybrid example	28
No silent historical reinterpretation	28
Documentation architecture	28
Canonical architecture manual	28
Formal CLI specification	28
User guide	29
Existing scientific manuals	29
Validation architecture	29
Unit-level contracts	29
Initial layer combination matrix	29
Duplicate-instance tests	29
Dependency-reuse tests	30
Worker/backend invariance	30
Renderer parity	30
Required benchmark scenes	30
Staged GFX3D implementation plan	30
GFX3D-1 - universal contracts and documentation	30

Scope	30
Non-goals	31
Acceptance	31
GFX3D-2 - decompose current graphical products	31
Scope	31
Required behavior	31
Acceptance	31
Implementation status - mdstats 0.20.147a0	31
GFX3D-3 - universal CLI and declarative configuration	32
Scope	32
Acceptance	32
Implemented authority (0.20.148a0)	32
GFX3D-4 - shared dependency planning and cache authority	32
Scope	32
Acceptance	33
Implemented authority (0.20.149a0)	33
GFX3D-5 - universal rendering and interaction semantics	33
Implemented authority (0.20.150a0)	33
Acceptance - satisfied	34
Post-foundation scientific visualization gates	34
GFX3D-1 detailed object model	34
Suggested module boundaries	35
Implementation source map for 0.20.145a0	36
Deliberate limitations	36
Acceptance criteria for the universal architecture	37
One-sentence architectural rule	37
Recommended next implementation	37
Revision record	37
2026-08-11 - GFX3D-5 implementation (0.20.150a0)	37
2026-08-11 - GFX3D-4 implementation (0.20.149a0)	38
2026-08-11 - GFX3D-3 implementation (0.20.148a0)	38
2026-08-11 - GFX3D-2 implementation (0.20.147a0)	38
2026-08-11 - GFX3D-1 implementation (0.20.146a0)	38
2026-08-11 - Initial universal GFX3D architecture	38
2026-08-11 - GFX3D-HARDEN1 implementation (0.20.156a0)	39
2026-08-12 - GFX3D-HARDEN2 implementation (0.20.157a0)	39
2026-08-12 - GFX3D-HARDEN3 implementation (0.20.158a0)	40
2026-08-12 - GFX3D-HARDEN4 implementation (0.20.159a0)	41

Document status

This document is the **single normative architecture standard and staged implementation plan** for the universal configurable three-dimensional graphics subsystem in mdstats.

The subsystem is named **GFX3D** in this manual. Its intended user-facing source-checkout launcher is:

```
python tools/mdstats-3d.py ...
```

and its intended package-level command surface may later be exposed as:

```
mdstats 3d ...
```

The CLI name is deliberately broad. GFX3D is not an atomic-density program and is not an LTA-specific program. It is the common scene-composition layer through which independent scientific products can be displayed together in one registered periodic three-dimensional view.

The immediate prototype is `examples/plot_lta_mixed_alkali_density.py` in `mdstats 0.20.145a0`. That script already combines:

- trajectory input and format resolution;
- LAMMPS type mapping and optional LAMMPS-log metadata;
- framework connectivity and topology construction;
- topology caching;
- atomic connectivity;
- framework registration;
- atomic trajectories;
- atomic mean connectivity;
- atomic densities;
- density resource policy and automatic execution planning;
- Plotly rendering;
- browser-mesh control;
- self-contained HTML output.

The current script is therefore no longer conceptually an example of one density function. It is a prototype **scientific scene composer**. GFX3D formalizes that role and removes the current LTA- and density-specific coupling.

The architecture distinguishes three statuses:

current baseline behavior implemented through `mdstats 0.20.150a0`, hardened in `0.20.156a0`, `0.20.157a0`, `0.20.158a0`, and `0.20.159a0`, and available for migration;

normative target behavior required by this standard but not yet implemented;

deferred behavior intentionally left to a later approved layer or renderer specification.

As of `mdstats 0.20.159a0`, **GFX3D-1 through GFX3D-5 are implemented and GFX3D-HARDEN1/HARDEN2/HARDEN3/HARDEN4 are complete**. The universal immutable request, selection, dependency, context, identity, registry, manifest, renderer-neutral primitive, generic render-result, and legacy compatibility contracts live under `mdstats.graphics3d`. GFX3D-2 adds registered independent `FrameworkTopologyLayer`, `AtomicConnectivityLayer`, `AtomicTrajectoryLayer`, and `AtomicDensityLayer` adapters. GFX3D-3 adds the packaged `mdstats-3d` command, strict TOML compilation, layer shorthand, the source-aware `lta-mixed-alkali-density` compatibility preset, canonical manifest-only mode, deterministic precedence/overwrite behavior, and the historical example compatibility shim. GFX3D-4 replaces the raw CLI's monolithic layer dependency with product-level framework/connectivity/trajectory/density keys plus single-flight shared cache authority. GFX3D-5 removes the remaining renderer coupling: layer adapters emit renderer-neutral primitives, the common Plotly backend dispatches only on primitive classes, and view state, periodic replication, visibility, render priority, generic legend grouping, and browser-payload accounting are universal render semantics rather than scientific-layer special cases.

The central design rule is:

A visualization is a renderer-independent scene composed of independent scientific layers. Layers declare data dependencies and rendering options; one scene planner computes each scientific dependency once, and renderers only materialize the prepared scene.

Source consolidation and authority

This manual consolidates and supersedes cross-cutting visualization-composition statements that are currently distributed across the plotting implementation and existing architecture/specification documents.

The principal current sources are:

Current source	Material incorporated into GFX3D
<code>examples/plot_lta_mixed_alkali_density.py</code>	prototype CLI workflow, trajectory input handling, resource options, current all-on LTA composition
<code>mdstats.plotting.framework_dynamics</code>	registered scene preparation, trajectory paths, mean framework, atomic mean graph, density integration, composite render result
<code>mdstats.plotting.graph_3d</code>	existing Plotly graph renderer, camera/cell controls, trace mapping, HTML export
<code>mdstats.plotting.framework_topology_graph</code>	framework topology graphical products
<code>mdstats.plotting.atomic_connectivity_graph</code>	atomic connectivity graphical products
<code>mdstats.plotting.atomic_density</code>	atomic density scientific products and density render controls
<code>mdstats.plotting.framework_density</code>	framework density products
<code>mdstats.plotting.periodic_graph</code>	periodic display replication and canonical-cell display semantics
<code>mdstats.plotting.graph_styles</code>	graph styling and chemistry-aware style resolution
Dynamical Framework and Density Architecture Standard	coordinate/registration invariants, density scientific identity, resource policy, renderer separation
MLFF campaign CLI architecture/specification pattern	thin source-checkout launcher, canonical configuration, explicit state/provenance contracts, separation between architecture/specification/user guide

This manual owns:

- the universal 3-D scene abstraction;
- the universal graphical-layer abstraction;
- scene/layer identity and provenance boundaries;
- the selection-language architecture;
- the shared scene-context and dependency-DAG model;
- layer registry and extension policy;
- generic layer visibility/order/interaction semantics;
- renderer-neutral prepared graphical primitives;
- the universal 3-D CLI configuration architecture;
- preset expansion rules;
- migration of the current hybrid LTA plot into a compatibility preset;
- the staged GFX3D implementation sequence and acceptance gates.

Scientific subsystem manuals remain authoritative for the meaning and construction of their own products. In particular:

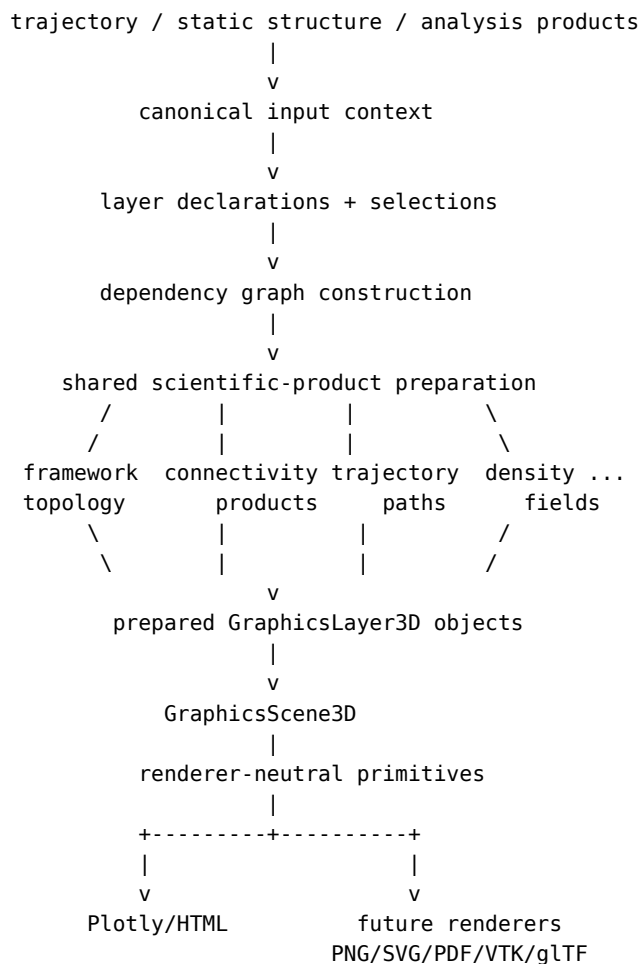
- density science remains owned by the Dynamical Framework and Density Architecture Standard;
- framework topology remains owned by the framework topology contracts;
- atomic connectivity remains owned by the atomic connectivity contracts;
- future ring/cage/site/kinetics manuals own the scientific definition of those objects.

GFX3D must not redefine those scientific objects merely to make them easier to render.

Requirements use **must**, **should**, and **may** in their usual normative sense. A gate cannot be passed by weakening a scientific identity, tolerance, resource limit, or determinism requirement without revising this standard and recording the rationale.

Architectural objective

The target architecture is a universal composition pipeline:



The architecture must allow the user to select **any arbitrary compatible combination of graphical layers**. No layer type is mandatory merely because the current prototype always displays it.

Examples of valid scenes include:

```
framework only
trajectory only
atomic connectivity only
Na density only
framework + trajectory
framework + atomic connectivity + density
Na trajectory + Na density
Na density + K density
framework + Na-0 connectivity + Na density
framework + 8R rings + sites + site transitions    [future]
site-density surface + Markov network                [future]
```

The architecture must also support multiple instances of one layer type. A type is not a singleton.

Design principles

Scientific ownership before graphics convenience

A graphical layer consumes or prepares a scientific product according to the owning subsystem's contract. GFX3D may coordinate that preparation, cache it, select from it, and render it. GFX3D may not silently alter the scientific definition in order to obtain a more convenient figure.

Independent layers, shared dependencies

Framework topology, atomic connectivity, trajectories, and atomic densities are separate graphical concepts. Their computations may share trajectory frames, registration, neighbor geometry, topology catalogs, and resource budgets. Shared dependencies are computed once by the scene planner and reused; the graphical layers remain independently selectable.

Scientific preparation is not rendering

Plotly, Matplotlib, VTK, or any future renderer must not determine:

- connectivity;
- topology;
- ring/cage identity;
- site identity;
- density estimator;
- density bandwidth;
- transition counting;
- state assignment;
- periodic registration;
- scientific frame selection.

Those decisions happen before rendering and are represented by immutable prepared objects and provenance.

Render options are not scientific options

Changing opacity, line width, color, camera direction, projection mode, legend grouping, or initial visibility must not invalidate the underlying scientific product.

Changing a density bandwidth, connectivity threshold, site-assignment policy, ring definition, frame set, or registration mode must invalidate the relevant scientific identity.

One coordinate gauge per scene

All spatial layers in one scene must refer to one declared display coordinate system and compatible display cell. A renderer may replicate canonical objects into periodic display images, but one layer may not independently recenter, rotate, or refold scientific coordinates relative to another layer without an explicit view transformation.

Display replication is not scientific duplication

Rendering a 2x2x2 periodic image, neighboring cages, or an outer-boundary cell is a view operation. It does not multiply occupancy, density normalization, trajectory weight, transition counts, or scientific evidence.

Arbitrary combinations must be first-class

The architecture is not considered compositional if only the old hybrid scene works. Every non-empty combination of the initial four layer families must be supported unless a scientifically explicit dependency makes that combination impossible.

Configuration must compile to one canonical scene declaration

CLI shorthand, TOML configuration, presets, and programmatic APIs must converge on the same canonical configuration objects. The implementation must not maintain separate scientific execution paths for “CLI mode”, “preset mode”, and “Python mode”.

Fail closed on ambiguity

An ambiguous LAMMPS type map, unknown species, duplicate layer name, unresolved site identifier, incompatible coordinate gauge, or missing required dependency is an error. GFX3D does not guess scientific meaning.

Visibility is not execution authority

A layer can be prepared but initially hidden so that an interactive viewer can enable it. Conversely, a layer absent from the scene must not be computed merely because an old preset used to include it.

Resource management is scene-wide

GFX3D inherits the runtime-derived CPU/RAM/GPU resource authorities already established in the density subsystem where applicable. Expensive dependencies owned by different layers must not independently assume they each own the complete machine.

Current baseline and architectural pressure

Current composite scene

`mdstats 0.20.145a0` exposes `FrameworkDynamicsScene` as a renderer-independent aggregate containing fixed named slots including:

```
mean_framework
trajectory_paths
atomic_mean_graph
frame_indices
weights
display_cell
options
resources
atomic_density_fields
framework_density_fields
planning_record
topology_categories
topology_catalog
dominant_topology_id
metadata
```

This is scientifically useful and should remain supported during migration, but it is not a scalable universal scene contract. Every future concept would otherwise require adding another fixed field such as:

```
rings
cages
sites
site_assignments
transition_paths
kinetic_network
free_energy_surfaces
force_vectors
...
```

The result would be a single ever-growing dataclass whose constructor and renderer know every science domain.

Current composite render result

`FrameworkDynamicsRenderResult` similarly contains fixed trace-index namespaces:

```
trajectory_trace_indices
atomic_mean_graph_trace_indices
endpoint_trace_indices
density_trace_indices
framework_density_trace_indices
```

This representation forces every future graphical element to modify the common result schema.

Current prototype CLI

`examples/plot_lta_mixed_alkali_density.py` currently decides in one script that an LTA plot will include:

- all present-species trajectories;
- an occupancy-filtered atomic mean graph;
- all detected atomic densities;
- mean framework topology;
- framework-registered coordinates;

- folded trajectories;
- fixed density science defaults;
- one Plotly output.

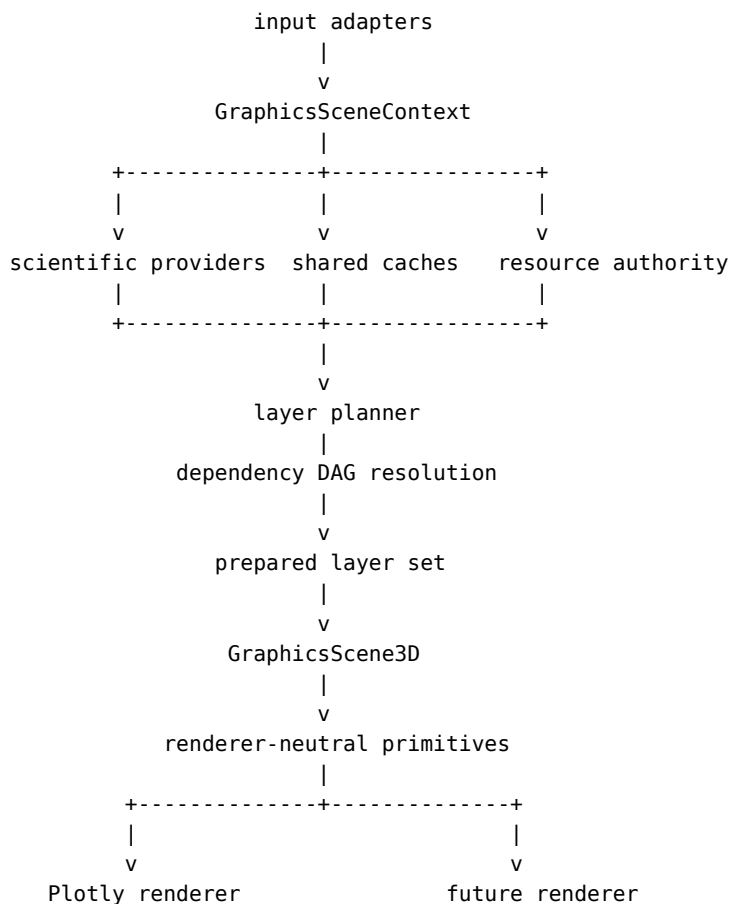
The script also owns useful generic behavior that should survive promotion:

- format inference across VASP XML, XDATCAR, concatenated CONTCAR trajectory, and LAMMPS dump input;
- LAMMPS metadata/type-map handling;
- topology cache input/output;
- resource controls;
- browser profile controls;
- self-contained HTML output;
- progress reporting.

The architectural task is therefore **decomposition and generalization**, not replacement of working science.

Dependency direction and ownership

The target dependency direction is:



The following dependency rules are normative:

1. Generic renderers must not import framework topology, atomic connectivity, density, ring, cage, site, or kinetics scientific modules.
2. Scientific providers must not import Plotly merely to prepare scientific objects.
3. Layer adapters may import their owning scientific provider and generic GFX3D contracts.
4. The scene planner may depend on the layer registry and dependency-provider registry, but not on specific layer types through a central if/elif chain.
5. The CLI may import configuration, input adapters, and the scene orchestrator. It must remain a thin launcher rather than the scientific implementation.

Terminology

scene one complete three-dimensional visualization request after canonical configuration resolution;

layer type a registered class of graphical content such as framework, connectivity, trajectory, or density;

layer instance one named occurrence of a layer type in a scene;

scientific dependency an immutable scientific product required to prepare one or more layers;

scene context the shared input, coordinate, cache, resource, and dependency state from which layers are prepared;

prepared layer a renderer-independent layer whose scientific dependencies and spatial data have been resolved;

render primitive backend-neutral geometric/display content such as point sets, polyline sets, triangle meshes, arrows, labels, or legend groups;

scientific identity digest/identity of the scientific content and policies that define a product;

render identity identity of graphical styling/view choices applied to prepared scientific content;

execution identity optional identity/evidence for worker counts, backends, timings, GPU decisions, or cache execution choices that do not define scientific content;

manifest normalized canonical representation of the complete scene request and resolved provenance;

preset a named configuration fragment expanded into ordinary canonical configuration; it is not a separate execution implementation.

Universal scene contract

Target object

The target public scientific composition object is conceptually:

```
GraphicsScene3D(  
    layers=(  
        FrameworkTopologyLayer(...),  
        AtomicConnectivityLayer(...),  
        AtomicTrajectoryLayer(...),  
        AtomicDensityLayer(...),  
    ),  
    context=...,  
    view=...,  
    resources=...,  
    metadata=...,  
)
```

The concrete implementation may separate declared and prepared scene records. The architecture recommends distinct records:

GraphicsScene3DRequest	# normalized user/scientific request
PreparedGraphicsScene3D	# immutable prepared scientific layers
Graphics3DRenderResult	# renderer-specific result + generic layer mappings

This avoids a request object gradually acquiring large numerical arrays and execution results.

Scene invariants

A prepared scene must satisfy:

- unique layer names;
- deterministic layer order;
- one declared coordinate/display gauge;
- compatible display-cell semantics for every spatial layer;
- normalized scene-level frame/time selection when applicable;
- every required dependency resolved;
- no undeclared scientific dependency created by the renderer;
- each prepared layer tied to scientific evidence/provenance;
- deterministic collation independent of parallel preparation order.

Layer order

Layer declaration order is the canonical default visual order and manifest order.

Dependency execution order is **not** visual order.

A renderer may need to group or batch primitives for technical reasons, but it must preserve the declared layer order in the generic layer-result mapping and interactive layer tree.

An explicit `render_priority` or equivalent may be introduced if a renderer requires an ordering distinct from declaration order. Such a field is nonscientific.

Universal layer contract

Layer instance identity

Every layer instance must have at least:

<code>name</code>	unique user-visible scene name
<code>type</code>	registered layer type identifier
<code>enabled</code>	whether the layer is part of the scene
<code>initially_visible</code>	interactive starting visibility
<code>selection</code>	scientific selection expression/options
<code>analysis</code>	scientific preparation options
<code>render</code>	nonscientific rendering options
<code>metadata</code>	user annotations / non-authoritative labels

`name` is not the scientific identity of the underlying data. Renaming `Na density` to `sodium occupancy` must not invalidate the `density` field.

Multiple instances

A scene may contain:

```
[[layer]] type="density" name="Na narrow"
[[layer]] type="density" name="Na broad"
[[layer]] type="density" name="K"
```

provided their names are unique and their scientific policies are explicit.

No layer registry may assume one instance per type.

Required interface

The exact Python protocol may evolve, but each registered layer adapter must conceptually provide:

```
layer_type() -> str
validate(request, context) -> None
dependencies(request) -> tuple[DependencyRequest, ...]
prepare(request, resolved_dependencies, context) -> PreparedGraphicsLayer3D
render_primitives(prepared, render_context) -> tuple[GraphicsPrimitive3D, ...]
```

The `render_primitives` step must not perform new expensive scientific analysis. It may perform view/display materialization such as periodic image replication or mesh simplification that is explicitly classified as rendering.

Required versus optional dependencies

Dependencies must be typed as required or optional.

Example:

```
SiteLayer
    required: SiteCatalog
    optional: CageCatalog      # used for labels/grouping if supplied
```

An optional dependency may enrich presentation but may not silently change the scientific identity of the required product unless the dependency is promoted to required and included in that identity.

Universal selection model

Purpose

The current code has several domain-specific selectors, including `TrajectoryAtomSelection` and `AtomicDensitySelection`. Future rings, cages, and sites would naturally add more. GFX3D requires one **selection architecture** so the CLI/configuration surface remains coherent even though individual layers support different selector fields.

Core selection vocabulary

The target selection representation must be able to express, where scientifically meaningful:

```
species == Na
species in [Li, Na, K]
atom_index in [1, 5, 9]
atom_id in [...]
pair == Na-0
pair in [Li-0, Na-0, K-0]
framework_role == vertex
ring_size == 8
ring_id in [...]
cage_type == alpha
cage_id in [...]
site_type == 8R
site_id in [...]
state_id == ...
topology_id == ...
transition == site_A -> site_B
```

The initial implementation need not implement a free-form expression parser. A typed canonical record is preferred first. For example:

```
GraphicsSelection(
    species=("Na",),
    atom_indices=(),
    pairs=(),
    topology_ids=(),
    ring_sizes=(),
    site_ids=(),
)
```

Layer validators define which fields are allowed.

Selection semantics

Selections are scientific when they determine which scientific entities contribute to the prepared layer. They therefore participate in layer scientific identity.

Render-only filtering performed after scientific preparation must be explicitly marked as render visibility filtering and may not be confused with scientific selection.

Stable identifiers

Index-based selections are fragile across transformed/generated objects. Where a subsystem exposes persistent IDs such as topology IDs, ring IDs, cage IDs, or site IDs, GFX3D should prefer those identifiers in manifests.

Ambiguity policy

The following must fail closed:

- a numeric LAMMPS type without a resolvable element mapping;
- an unknown species symbol;
- a requested pair not represented in the selected connectivity definition when the layer requires it;
- an unknown topology/ring/cage/site ID;
- one textual label resolving to multiple scientific IDs without explicit disambiguation.

Scene context and dependency DAG

Shared scene context

The target GraphicsSceneContext owns shared state such as:

- source trajectory / static structure
- source identities and metadata
- resolved atom/species map
- scene frame/time selection
- frame weights
- reference/display cell
- registration transform/gauge
- runtime resource budget
- progress port
- scientific product cache
- periodic neighbor geometry cache
- topology catalog
- atomic connectivity states
- registered coordinate cache
- density scheduler
- future ring catalog
- future cage catalog
- future site catalog
- future site assignments
- future transition network

Not every scene populates every entry.

Dependency keys

A scientific dependency cache key must contain all scientific policies that affect the product and exclude render-only/execution-only options.

Examples:

```
RegisteredCoordinatesKey(  
    source_id,  
    frame_selection_id,  
    registration_policy_id,  
    reference_frame_id,  
)  
  
AtomicConnectivityKey(  
    source_id,  
    frame_selection_id,  
    connectivity_definition_id,  
)  
  
AtomicDensityKey(  
    registered_coordinates_id,  
    selection_id,  
    density_operator_id,  
    grid_policy_id,  
    bandwidth_policy_id,  
)
```

DAG construction

Layer declarations are translated into dependency requests. Equal dependency keys are deduplicated before execution.

A conceptual scene may produce:

```
trajectory source  
|
```

```

+--> registered coordinates -----+
|                                   |
+--> framework connectivity --> topology -----+--> framework layer
|                                   |
+--> full atomic connectivity -----> connectivity layer
|                                   |
+-----+--> trajectory layer
|                                   |
+--> density field --> density layer

```

Future site/kinetics extension:

```

framework topology --> rings --> cages -----+
registered ion trajectories --> site discovery ---+--> site catalog
site catalog + trajectories --> assignments --> transitions --> Markov layer

```

Parallel dependency execution

Independent dependency nodes may execute concurrently under one scene-wide resource authority. Completion order must not change manifest ordering, layer ordering, scientific identity, or numerical results.

Cache scope

Caches may be:

- in-memory scene-local;
- reusable process-local;
- durable content-addressed caches when the owning subsystem authorizes persistence.

Cache location is execution metadata, not scientific identity.

Scientific, render, and execution identity

GFX3D requires three explicit identity domains.

Scientific identity

Scientific identity covers facts that define the prepared scientific object. Examples:

- source content ID;
- selected frames and weights;
- species/atom/site selection;
- connectivity definition;
- topology identity;
- registration policy;
- density operator, grid, bandwidth, and HDR definition;
- ring/cage/site definitions;
- transition counting policy.

Changing any of these changes the relevant scientific identity.

Render identity

Render identity covers appearance and view materialization, including:

- colors;
- line widths;
- opacities;
- marker shapes and sizes;
- camera;
- projection mode;
- initial visibility;
- legend title/grouping;
- periodic display replication;

- browser mesh profile where it is a display approximation that does not alter the scientific field.

Execution identity

Execution identity/evidence covers:

- worker count;
- CPU affinity;
- GPU device;
- dense versus sparse storage when scientifically equivalent under an approved operator;
- direct versus FFT execution partition when frozen as execution authority;
- timings;
- memory usage;
- cache hit/miss state;
- process/thread executor choice.

The exact ownership of backend identity follows the owning scientific subsystem. Where an execution choice is scientifically equivalent and explicitly designated backend-neutral, GFX3D must not promote it into scene scientific identity.

Canonical scene manifest

Purpose

After CLI/TOML/preset parsing and before expensive scientific preparation, GFX3D must materialize a normalized canonical scene manifest.

The manifest plays a role analogous to the immutable campaign configuration/provenance records in the MLFF CLI, but it is not necessarily an approval gate for ordinary plotting.

Manifest content

At minimum the manifest should record:

```

schema_version
mdstats_version
source descriptors and content identities
resolved input format
resolved atom/species mapping
scene frame/time selection
coordinate/registration policy
display-cell policy
resolved resource request
ordered canonical layer requests
expanded presets
selection records
scientific option records
render option records
view/camera record
output request
layer dependency keys after planning
warnings/degradations

```

After preparation, a result manifest/evidence record may additionally include:

```

resolved scientific dependency identities
prepared layer scientific IDs
render IDs
execution IDs/evidence
cache reuse summary
resource report
output artifact hashes

```

Manifest serialization

Canonical JSON is recommended for machine evidence. TOML remains the user configuration format.

The user should be able to request the normalized configuration/manifest without rendering, for example through a future command such as:

```
python tools/mdstats-3d.py scene.toml --print-manifest
```

Exact CLI command shape belongs to the later CLI specification, not this architecture manual.

Layer registry and extension model

Registry requirement

Layer types must be registered rather than hard-coded in one central dispatcher.

Conceptually:

```
register_graphics_layer("framework", FrameworkTopologyLayerAdapter)
register_graphics_layer("connectivity", AtomicConnectivityLayerAdapter)
register_graphics_layer("trajectory", AtomicTrajectoryLayerAdapter)
register_graphics_layer("density", AtomicDensityLayerAdapter)
```

Future additions become:

```
register_graphics_layer("ring", RingLayerAdapter)
register_graphics_layer("cage", CageLayerAdapter)
register_graphics_layer("site", SiteLayerAdapter)
register_graphics_layer("transition", TransitionPathLayerAdapter)
register_graphics_layer("markov", MarkovNetworkLayerAdapter)
```

Internal-first plugin policy

GFX3D-1 only requires an internal registry. A public third-party plugin API is deferred until the built-in contracts stabilize.

The internal registry must nevertheless avoid assumptions that would make a future plugin API impossible.

Registry metadata

Each layer type should advertise:

- canonical type name
- schema/version
- supported selection fields
- scientific option schema
- render option schema
- required dependency providers
- optional dependency providers
- supported renderer primitive classes

Initial layer family: framework topology

Scientific ownership

Framework topology remains owned by the framework-topology scientific contracts and TopologyCatalog/FrameworkTopology products.

Layer purpose

FrameworkTopologyLayer renders a framework topology view independently of atomic connectivity.

Potential modes include current capabilities such as:

- mean / dominant framework
- selected topology category
- projected framework view
- atomic-path diagnostic view

Mode names must be aligned with the owning framework-topology contracts during GFX3D-2.

Dependencies

Typical dependencies:

trajectory/static structure
framework mapping
framework connectivity or supplied topology catalog
registration/display gauge
mean/category framework geometry

If a fully prepared framework graph is supplied programmatically, lower dependencies may be bypassed through an explicit prepared-product input contract.

Render controls

Render controls include node/edge styling, legend visibility, labels, and opacity. They do not alter framework topology identity.

Initial layer family: atomic connectivity

Scientific distinction

Atomic connectivity is **not** framework topology.

Framework topology describes the structural framework graph or its projected/atom-resolved topological representation. Atomic connectivity describes actual atom-atom contacts under a declared connectivity definition and may include mobile-ion coordination.

The universal scene must allow both to coexist.

Layer modes

Initial GFX3D decomposition should support the current averaged/occupancy representation. The architecture allows future modes such as:

selected-frame connectivity
mean/occupancy connectivity
state-conditioned connectivity
transition-highlighted connectivity

Dependencies

Typical dependencies:

registered atomic coordinates
AtomicConnectivityResult
optional frame weights
mean periodic positions / occupancy reduction

Pair selection

Pair filtering such as Na-0 or K-0 is scientific selection when it determines which connectivity edges are part of the layer.

Initial layer family: atomic trajectory

Scientific product

The initial layer consumes prepared atomic trajectories equivalent to current TrajectoryPathSet products.

Selection

The layer supports atom/species selection through the universal selection model.

Path modes

Current continuous and folded path modes remain scientifically meaningful preparation choices and should migrate into the trajectory layer's analysis options.

Registration

Trajectory registration must use the scene's shared registered coordinates. A trajectory layer may not independently perform a conflicting registration merely because a different visual view is desired.

Render options

Line width, opacity, grouping by species, endpoint marker visibility, hover, and legend behavior are render options.

Multiple trajectory layers are allowed.

Initial layer family: atomic density

Scientific ownership

Atomic density remains owned by the density subsystem. GFX3D delegates scientific field creation to the existing qualified density pipeline rather than creating a new density estimator.

Layer request

One density layer instance contains:

```
selection
density scientific options
HDR/display-shell request
render options
```

Where HDR thresholds are scientifically derived from a density field, the threshold definition belongs to the scientific/display-preparation evidence as already defined by the density subsystem. Opacity and color remain render-only.

Multiple densities

The architecture must support multiple independent density layers, including multiple selections and, when explicitly requested, multiple scientific bandwidth definitions.

For example:

```
Na density, sigma policy A
K density, sigma policy A
Na density, sigma policy B
```

Each scientifically distinct field has its own scientific identity.

Resource integration

Density layers inherit the PAR-DENS0-PAR-DENS6 execution/resource architecture. Multiple density layer requests should be planned together where the density subsystem can gain from whole-scene scheduling.

GFX3D must not instantiate a separate uncontrolled density scheduler per layer.

Framework density and other current density products

The current FrameworkDynamicsScene can also carry framework vertex/edge density fields. GFX3D should generalize density layer typing sufficiently that these can be represented without restoring a special composite scene.

Two approaches are acceptable:

1. a common DensityLayer3D with a scientific source kind (atomic, framework_vertex, framework_edge); or
2. separate registered layer types sharing a common density-render adapter.

The choice should be made in GFX3D-2 based on actual type/provenance reuse. The user-facing CLI should not expose implementation inheritance details.

Future layer families

The architecture is intentionally designed before the following scientific products are fully implemented.

Ring layer

Potential graphical products:

- ring polygons;
- ring centers;
- ring normals;
- ring labels;
- primitive/equivalence classification;
- selected ring-size families such as 4R, 6R, 8R.

Scientific ring discovery remains outside the renderer.

Cage layer

Potential products:

- cage boundary surfaces/wireframes;
- cage centers;
- alpha/beta or other cage labels;
- cage membership highlighting;
- natural-tiling adjacency.

Site layer

Potential products:

- discovered site centers;
- uncertainty/extent markers;
- site family labels such as 4R/6R/8R;
- side/off-center variants;
- occupancy-scaled markers;
- assignment-quality annotations.

Site-assignment layer

A separate layer may display current or time-aggregated ion-to-site assignment, transition frames, or classification confidence without conflating those products with the site catalog itself.

Transition-path layer

Potential products:

- transition trajectories between sites;
- arrows;
- directional frequency/flux encodings;
- selected transition classes.

Markov/kinetics layer

A three-dimensional site-kinetics network may render:

- nodes at site centers;
- directed edges for transitions;
- line width mapped to counts/rates;
- color mapped to barriers/free energies/rates;
- labels containing state or kinetic information.

Its scientific inputs are a site catalog, state assignments, and a transition/Markov model. The renderer does not estimate rates.

Free-energy and vector layers

The registry should later be able to support free-energy/PMF surfaces, local force vectors, polarization vectors, phonon eigenvectors, or other spatial scientific products without changing GraphicsScene3D itself.

Coordinate system, registration, and display gauge

Scene-level authority

The scene owns the default coordinate/registration policy. Initial modes should reuse the scientifically qualified concepts already present in FrameworkDynamicsOptions, including material, laboratory, and framework-registered coordinates where applicable.

Layer compatibility

A layer may require a specific coordinate product. If that product cannot be represented in the scene gauge without loss of its scientific meaning, scene validation must fail or require an explicit subscene/inset architecture. Silent per-layer coordinate drift is prohibited.

Display cell

The scene declares reference/mean display-cell policy or another future explicitly specified cell policy.

Every prepared spatial layer records the display cell against which its coordinates are interpreted.

Camera/view transform

Camera rotation, perspective/orthographic projection, zoom, and fit-to-cell are nonscientific view operations.

Future named view presets may include:

```
[100]
[110]
[111]
ring-normal
cage-centered
fit-to-cell
```

A view preset must not modify scientific coordinates.

Periodic display and replication

GFX3D should reuse the existing periodic_graph/CanonicalCellDisplay concepts rather than implementing periodic display separately for each layer.

The scene/view layer may request:

```
canonical/reference cell only
outer boundary
explicit image ranges
neighboring periodic replicas
```

Periodic replication must operate on prepared display primitives or on a shared periodic-materialization service.

Scientific products remain canonical. The renderer must not multiply scientific normalization or counts when images are displayed.

Renderer-neutral graphical primitives

Motivation

The universal scene representation must not be a collection of Plotly traces. Otherwise the architecture remains Plotly-specific even if class names become generic.

Initial primitive set

A minimal renderer-neutral primitive vocabulary should support:

PointSet3D
PolylineSet3D
SegmentSet3D
TriangleMesh3D
ArrowSet3D
TextLabelSet3D
CellWireframe3D
LegendGroup

The exact class names may change. The important property is that they contain geometry, per-element display attributes, hover/label metadata, and provenance references without depending on Plotly objects.

Primitive provenance

Every primitive group must identify its owning layer instance and, where meaningful, the scientific source IDs from which it was prepared.

Renderer role

A renderer converts primitives to backend objects, manages backend trace/object budgets, and writes artifacts. It does not modify scientific products.

Renderer interface

The target generic renderer interface is conceptually:

```
class Graphics3DRenderer:
    def render(
        self,
        scene: PreparedGraphicsScene3D,
        *,
        options: Graphics3DRenderOptions,
    ) -> Graphics3DRenderResult: ...
```

The first implementation remains Plotly-backed.

Future renderers may target:

- static raster output;
- SVG/PDF-capable vector/static output;
- VTK;
- glTF;
- other interactive backends.

No future renderer is required by GFX3D-1 through GFX3D-5. The architecture merely prevents the first implementation from making Plotly the scientific scene model.

Generic render result

The universal result should replace fixed trace namespaces with layer-keyed results:

```
Graphics3DRenderResult(
    artifact=...,
    scene=...,
    layer_results={
        "framework": LayerRenderResult(...),
        "Na-0 coordination": LayerRenderResult(...),
        "Na trajectory": LayerRenderResult(...),
        "Na density": LayerRenderResult(...),
    },
    render_metadata=...,
    warnings=...,
)
```

Each LayerRenderResult may contain:

backend object/trace indices
legend group
initial visibility
scientific evidence IDs
render identity
execution/render metadata
warnings
resource/payload contribution

Renderer-specific indices are allowed inside the renderer-specific result, but the top-level schema does not gain a new field for every layer type.

Legend, visibility, and interaction model

Generic layer tree

Interactive renderers should expose a generic layer organization rather than bespoke controls such as “show density” or “show trajectory”.

Each named layer becomes an independently addressable visibility group.

Nested grouping may be introduced, for example:

```
Framework
Connectivity
  Na-0
  K-0
Trajectories
  Na
  K
Densities
  Na
  K
Sites
Transitions
```

Initial visibility

`initially_visible` is a render setting. Hidden layers remain part of the prepared scene and can be toggled without recomputation in a self-contained interactive artifact.

Layer opacity

A generic interactive opacity control may be supported only when the renderer can apply it without changing scientific representation. Layer-specific render schemas remain authoritative.

Legend identity

Legend entries are tied to named layer instances and subgroups rather than to hard-coded scientific family names in the common renderer.

View presets and reproducible snapshots

A publication or diagnostic figure should be reproducible not only scientifically but visually.

The scene configuration should allow a view record containing:

```
[view]
projection = "orthographic"
camera_eye = [1.2, 1.2, 0.9]
cell_mode = "reference"
visible_layers = ["framework", "Na density", "Na sites"]
```

A renderer may also export a snapshot/view-state record from an interactive session in a later stage.

View state is render provenance and must not contaminate the scientific IDs of layers.

CLI architecture

Thin launcher

The source-checkout launcher should follow the MLFF CLI pattern:

```
tools/mdstats-3d.py
```

It bootstraps the adjacent source tree when run from a checkout, imports the packaged CLI implementation, and returns a deterministic UNIX exit status.

The substantive implementation belongs under the package, for example:

```
mdstats/graphics3d/  
mdstats/cli/graphics3d.py
```

Exact package paths may be refined during GFX3D-1.

CLI role

The CLI is responsible for:

- locating and parsing configuration;
- applying CLI overrides/shorthand;
- resolving input adapters;
- expanding presets;
- materializing the canonical scene manifest;
- invoking validation/preparation/rendering;
- emitting progress and user-facing diagnostics;
- writing requested artifacts.

The CLI is not responsible for implementing scientific analysis.

Source formats

The initial CLI should preserve the trajectory formats already supported by the prototype where the corresponding readers are available:

```
VASP vasprun.xml  
VASP XDATCAR  
watcher/custom concatenated CONTCAR TRAJECTORY  
LAMMPS custom dump / lammprj
```

The architecture should allow static POSCAR/CONTCAR/CIF/LAMMPS-data inputs for static graphical layers once the generic input adapters are wired, but GFX3D-1 need not require every format on day one.

Declarative configuration

TOML as the complex-scene format

Complex scenes should use TOML rather than accumulating dozens of command-line flags.

A representative target configuration is:

```
[scene]  
title = "Na-LTA 300 K"  
registration = "framework_registered"  
display_cell = "reference"  
output = "na_lta.html"  
  
[input]  
path = "dump.prod.Na_lta_300K.lammprj"  
format = "lammps-dump"  
lammps_type_map = "1=Si,2=Al,3=O,4=Na"  
  
[resources]
```

```

max_threads = "auto"
max_memory = "auto"

[[layer]]
type = "framework"
name = "framework"

[layer.analysis]
mode = "mean"

[layer.render]
node_size = 4.0
edge_width = 1.5
edge_opacity = 0.45

[[layer]]
type = "connectivity"
name = "Na-0 coordination"

[layer.selection]
pairs = ["Na-0"]

[layer.analysis]
mode = "occupancy"
occupancy_threshold = 0.95

[[layer]]
type = "trajectory"
name = "Na trajectories"

[layer.selection]
species = ["Na"]

[layer.analysis]
path_mode = "folded"

[layer.render]
opacity = 0.25
line_width = 1.5

[[layer]]
type = "density"
name = "Na density"

[layer.selection]
species = ["Na"]

[layer.analysis]
grid_interval = 0.20
adaptive_smearing = true

[layer.render]
mass_fractions = [0.50, 0.80, 0.95]
inner_opacity = 0.22
outer_opacity = 0.04

```

The exact field names become normative only in the later CLI specification. This example establishes the architectural shape.

CLI shorthand

Simple scenes may use shorthand such as:

```
--layer framework
--layer trajectory:Na
--layer density:Na
```

Shorthand must compile into the same canonical layer request records as TOML. It is not a second parser-to-science route.

Precedence

The later CLI specification must define deterministic precedence among:

1. built-in defaults;
2. preset expansion;
3. configuration file;
4. explicit CLI overrides.

The canonical manifest records resolved values and provenance of overrides where useful.

Preset architecture

Presets are configuration

A preset is a named configuration fragment, not a specialized code path.

Examples may include:

```
lta-mixed-alkali-density
framework-topology
atomic-connectivity
site-discovery
site-kinetics
```

The current `plot_lta_mixed_alkali_density.py` behavior should become the first compatibility preset.

Legacy preset

The compatibility preset should reproduce the current prototype's intended defaults as closely as practical:

- present-species trajectories;
- present-species atomic densities;
- mean framework;
- occupancy atomic mean connectivity;
- framework-registered coordinates;
- folded paths;
- current density defaults;
- balanced browser mesh profile;
- self-contained Plotly HTML.

LTA-specific topology/connectivity calibration belongs in an LTA preset/provider, not in the universal scene core.

Resource architecture

One authority

GFX3D must not weaken the resource policy established by the density work.

A scene has one resolved runtime resource budget. Scientific providers that already expose qualified schedulers, such as density preparation, receive compatible scoped budgets from the scene context.

Cross-layer admission

As the system grows, expensive independent products may include:

- topology/connectivity reconstruction;
- density preparation;
- ring/cage construction;

- site discovery;
- mesh preparation.

The scene planner should eventually admit independent dependency tasks under aggregate CPU/RAM/GPU constraints rather than launching each provider at its maximum independently.

GFX3D-1 only needs the contracts; broad cross-subsystem scheduling may be implemented incrementally.

Browser resources

Browser payload/trace/mesh budgets remain rendering constraints. They may alter display approximations only where the owning subsystem declares such approximation scientifically/display neutral. They may not coarsen scientific density fields or drop requested scientific layers silently.

Progress and diagnostics

The CLI should reuse the package progress-port abstraction.

Progress phases should describe architectural work rather than implementation internals. Examples:

```
input
manifest
scene_dependencies
registration
framework_topology
atomic_connectivity
density_preparation
layer_preparation
rendering
output
```

Parallel dependency work may produce nested progress events, but final reporting remains deterministic.

Warnings should be attached to the relevant dependency/layer where possible and also summarized at scene level.

Failure semantics

Errors should be grouped conceptually into:

configuration error invalid TOML, duplicate names, unsupported layer options, ambiguous selectors;
input error unsupported or inconsistent trajectory/static data, unresolved species/type map;
dependency error missing topology/site catalog or incompatible dependency policy;
scientific preparation error owning subsystem rejects the requested scientific analysis;
resource error request cannot be admitted under declared CPU/RAM/GPU/browser budgets without changing science;
render error renderer/backend cannot materialize an otherwise valid prepared scene;
output error artifact cannot be written.

The CLI must not catch a scientific error and silently remove the affected layer unless the user explicitly requested a best-effort/degraded mode defined by a future specification.

Output architecture

Initial output

The first GFX3D renderer remains self-contained Plotly HTML.

Sidcar evidence

The CLI should support writing a sidcar canonical manifest/result record, for example:

```
figure.html
figure.scene.json
```

The exact default naming belongs to the CLI specification.

Future outputs

The architecture should allow:

HTML
static PNG
SVG/PDF through an appropriate renderer/exporter
scene JSON/manifest
VTK
glTF

No future format is required for the first implementation gates.

Compatibility and migration

FrameworkDynamicsScene

FrameworkDynamicsScene remains readable and renderable during migration. It becomes a **legacy composite compatibility facade**, not the target scene API.

A compatibility adapter should be able to translate one FrameworkDynamicsScene into a PreparedGraphicsScene3D containing the equivalent built-in layers.

FrameworkDynamicsRenderResult

The existing result remains supported for old APIs. New GFX3D code uses generic layer-keyed results.

plot_framework_dynamics_3d

The existing function may remain as a compatibility wrapper that constructs/uses the GFX3D Plotly renderer internally once parity is established.

Hybrid example

examples/plot_lta_mixed_alkali_density.py should not remain the primary implementation.

Migration target:

1. move generic workflow behavior to package-level GFX3D code;
2. add tools/mdstats-3d.py thin launcher;
3. represent the old LTA workflow as a named preset/configuration provider;
4. leave the example for one compatibility cycle as a thin wrapper or tutorial pointing to the new CLI;
5. later remove duplicated implementation once deprecation policy allows.

No silent historical reinterpretation

Existing scene/density scientific evidence retains its historical schema and identity. GFX3D adapters may reference it; they do not rewrite it into new scientific evidence merely to fit the universal scene schema.

Documentation architecture

GFX3D should follow the successful MLFF documentation separation.

Canonical architecture manual

This document:

docs/arch_manuals/mdstats_3d_graphics_architecture.md
docs/arch_manuals/mdstats_3d_graphics_architecture.pdf

owns universal scene/layer/dependency architecture.

Formal CLI specification

A later GFX3D gate should create:

docs/specs/graphics3d/mdstats_gfx3d_cli_spec.md
docs/specs/graphics3d/mdstats_gfx3d_cli_spec.pdf

It owns exact command syntax, TOML fields, precedence, exit codes, manifest schema, and CLI acceptance tests.

User guide

A concise practical guide should be created when the CLI is executable:

docs/guides/mdstats_3d_graphics_user_guide.md
docs/guides/mdstats_3d_graphics_user_guide.pdf

It answers how to construct common figures and does not duplicate the architecture.

Existing scientific manuals

The Dynamical Framework and Density Architecture Standard should be revised during GFX3D implementation to state that:

- density scientific preparation remains owned there;
- FrameworkDynamicsScene is the compatibility composite scene;
- generic composition/rendering ownership moves to GFX3D;
- density-specific HDR/mesh scientific/display semantics remain in the density manual.

Future ring/cage/site/kinetics manuals should similarly point to GFX3D for composition rather than defining their own universal renderer.

Validation architecture

Unit-level contracts

Tests must cover:

- unique layer names;
- deterministic layer ordering;
- layer registry behavior;
- selection validation;
- dependency-key equality/deduplication;
- required/optional dependency behavior;
- scientific/render/execution identity separation;
- manifest canonicalization and round-trip;
- renderer-independent primitive contracts.

Initial layer combination matrix

With four initial layer families:

F = framework
C = atomic connectivity
T = trajectory
D = density

all 15 non-empty combinations must be tested:

F C T D
FC FT FD CT CD TD
FCT FCD FTD CTD
FCTD

A combination passes only if omitted layers are genuinely not required/computed unless another requested layer scientifically depends on the same underlying product.

Duplicate-instance tests

Tests must include:

density:Na + density:K
trajectory:Na + trajectory:K
connectivity:Na-0 + connectivity:K-0
multiple density definitions for one species

Dependency-reuse tests

At minimum verify that:

- multiple registered layers share one registered-coordinate product;
- framework and atomic-connectivity layers reuse compatible periodic neighbor geometry where authorized;
- multiple density layers share the existing whole-scene density resource authority;
- cache reuse does not alter scientific IDs.

Worker/backend invariance

Where owning subsystems already guarantee worker/backend invariance, GFX3D composition must preserve it. Scene/layer ordering, worker allocation, or rendering order may not perturb scientific outputs.

Renderer parity

The compatibility LTA preset must reproduce the existing hybrid example within the current graphical/scientific tolerances:

- same scientific framework/topology product;
- same connectivity product;
- same trajectories;
- same density field identities;
- same HDR thresholds;
- equivalent periodic display;
- equivalent browser budget;
- equivalent user-visible content, allowing intentional legend/layer-control redesign.

Required benchmark scenes

GFX3D should retain a small deterministic fixture and one real trajectory qualification case.

For the current LTA branch, the already-used Na-LTA MLFF trajectory is a strong real-data benchmark:

10,001 frames
168 atoms
type map: 1=Si, 2=Al, 3=O, 4=Na

The GFX3D benchmark should exercise a configurable subset of:

framework
Na-0 connectivity
Na trajectory
Na density
Si/O density where appropriate

This benchmark is not LTA as a hard-coded architecture dependency; it is a demanding integration fixture for the initial implementation.

Staged GFX3D implementation plan

The implementation must be gated. Do not collapse the entire migration into one rewrite.

GFX3D-1 - universal contracts and documentation

Scope

Introduce the universal scene foundation without changing scientific algorithms.

Required deliverables:

- GraphicsScene3DRequest or equivalent normalized scene request;
- GraphicsLayer3D request/prepared-layer contracts;
- unique named layer instances;
- universal selection record;
- dependency request/key contracts;

- GraphicsSceneContext foundation;
- scientific/render/execution identity separation;
- layer registry;
- canonical manifest schema;
- renderer-neutral primitive contracts;
- compatibility adapters for existing scene/result types where practical;
- this architecture manual installed in the package;
- initial skeleton of the formal CLI specification.

Non-goals

GFX3D-1 does not need to fully decompose every existing layer implementation or expose the final CLI.

Acceptance

- contracts serialize deterministically;
- duplicate layer names fail;
- layer registry is deterministic;
- equivalent dependency requests deduplicate;
- changing render-only options does not change scientific layer identity;
- changing scientific selection/options does;
- current package tests remain green.

GFX3D-2 - decompose current graphical products

Scope

Implement the initial independent layer adapters:

```
FrameworkTopologyLayer
AtomicConnectivityLayer
AtomicTrajectoryLayer
AtomicDensityLayer
```

Migrate current scientific products without changing their definitions.

Required behavior

- each layer can be requested alone;
- arbitrary compatible combinations work;
- multiple instances per type work;
- shared dependencies are reused;
- Plotly output remains available through the common renderer;
- FrameworkDynamicsScene can be adapted to the new prepared scene.

Acceptance

All 15 non-empty initial layer combinations pass, plus duplicate-instance tests.

Implementation status - mdstats 0.20.147a0

GFX3D-2 is implemented. The default registry now carries the four initial production adapters. `prepare_graphics3d_scene()` resolves their shared prepared-scene dependency once through `GraphicsSceneContext`, and `render_graphics3d_plotly()` composes arbitrary prepared layer instances into a generic `Graphics3DRenderResult` with non-overlapping layer-owned trace indices. Current trajectory and connectivity selections are filtered from already-prepared products; density field selection is fail-closed and is proven from prepared provenance/atom identities rather than guessed from labels. The common Plotly backend reuses the qualified legacy rendering routines without recomputing framework, connectivity, trajectory, or density science.

Focused qualification covers all 15 non-empty combinations of framework/connectivity/trajectory/density, duplicate trajectory and density instances, pair-filtered connectivity, absent-density fail-closed behavior, GFX3D-1 contracts, and the legacy framework/graph renderer surface. The focused result is 62 passed.

GFX3D-3 - universal CLI and declarative configuration

Scope

Promote the prototype workflow into the universal CLI.

Required deliverables:

tools/mdstats-3d.py
package-level CLI implementation
TOML scene configuration
layer shorthand
preset expansion
canonical manifest output
input-format handling migrated from the prototype
progress/error reporting
self-contained HTML output
formal CLI specification
short user guide

The existing hybrid LTA behavior becomes a compatibility preset.

Acceptance

- TOML and equivalent CLI shorthand compile to identical canonical requests;
- configuration precedence is deterministic;
- the old hybrid workflow is reproducible through the new CLI;
- omitted trajectory/connectivity/density products are not prepared unless requested or scientifically required by the current compatibility provider;
- CLI exits fail closed on ambiguous input/selection.

Implemented authority (0.20.148a0)

GFX3D-3 is implemented. `mdstats.graphics3d.cli` is the packaged command authority and `tools/mdstats-3d.py` is only a source-tree launcher. TOML, the `lta-mixed-alkali-density` preset, and repeated `--layer TYPE[:SELECTOR] [@NAME]` shorthand all compile into the same immutable `GraphicsScene3DRequest`. Precedence is fixed as defaults `< preset < TOML < explicit CLI`, with a stronger replacement rule for layer lists: TOML `[[layer]]` replaces preset layers, while any explicit `--layer` list replaces both.

The command supports VASP XML/XDATCAR/concatenated-CONTCAR trajectories and native LAMMPS dumps, including explicit LAMMPS units/timestep/type mapping and topology overrides. `--manifest-only` resolves the source and canonical request without framework/connectivity/density preparation. Normal execution writes both a self-contained Plotly HTML artifact and a canonical scene-manifest JSON. Existing output/manifest paths are protected unless `--force` is explicit. Unknown TOML keys and ambiguous scientific selections fail closed.

The historical `examples/plot_lta_mixed_alkali_density.py` is now a compatibility shim around the same command authority. When invoked without a config, preset, or explicit layers, it injects the `lta-mixed-alkali-density` preset and preserves the historical default HTML filename. The migrated LTA scientific provider retains the previous T-O-T framework mapping, hysteretic cutoff calibration, mobile-ion/O connectivity, framework registration, trajectory, and density defaults. Under GFX3D-4 these products are exposed through separate dependency keys; the current Plotly compatibility backend may still retain a temporary prepared-scene reference until GFX3D-5 removes renderer coupling.

GFX3D-4 - shared dependency planning and cache authority

Scope

Generalize the shared scene context into a complete dependency DAG and eliminate redundant whole-trajectory work across layers.

Required work includes, where applicable:

- shared registration;
- shared frame/weight resolution;

- shared topology/connectivity products;
- periodic-neighbor geometry reuse;
- density scene scheduling reuse;
- explicit durable/in-memory cache policy;
- cross-layer resource admission;
- dependency-level progress and timing evidence.

Acceptance

- duplicate dependency requests execute once;
- parallel dependency execution is deterministic;
- resource limits are obeyed scene-wide;
- scientific identities are independent of cache location/hit state;
- real Na-LTA benchmark shows reduced redundant work without changing results.

Implemented authority (0.20.149a0)

GFX3D-4 is implemented. Built-in raw-source layers now request one of four scientific product providers: `framework_topology_product`, `atomic_connectivity_product`, `atomic_trajectory_product`, or `atomic_density_product`. Equal keys deduplicate before execution. `GraphicsSceneContext.resolve_dependency()` uses a single-flight cache, so concurrent requests for the same scientific key execute exactly one resolver; waiters consume the stored result. Cache hits, waits, failures, resolver wall time, and provider execution records are execution evidence only and do not enter `GraphicsDependencyKey.identity`.

The current LTA source is represented by `LTAGraphics3DDependencySource`. It retains the existing framework/connectivity/density algorithms as the scientific owner and may batch jointly qualified preparation once, but exposes the resulting products separately to the scene DAG. Thus a framework + trajectory + density scene has three dependency keys, while adding connectivity produces four; duplicate instances of one layer family share their product dependency. Manifest-only mode constructs these product keys without scientific preparation.

Dependency resolution may begin concurrently, but result collation follows dependency-plan order. The current LTA provider serializes its one joint scientific preparation under the existing framework-dynamics CPU/RAM scheduler and then serves each product, so dependency parallelism cannot create a second resource authority. Durable scientific product caching is deliberately not authorized yet: GFX3D-4 cache policy is explicit in-memory single-flight reuse for one scene context, and durable cache location/hit state is excluded from scientific identity.

On the supplied Na-LTA trajectory (10,001 frames x 168 atoms), the authenticated source SHA-256 is:

```
81c86cc40f5a11031f80817213eb558c02348494d1c6cad9b4775a5bc3c9f9cd
```

A stride-500 four-family qualification resolves four product dependencies from one source scientific preparation (`preparation_count = 1`). A framework + Na trajectory + Na density scene resolves exactly three dependencies and omits connectivity. Against 0.20.148a0, mean-framework coordinates, Na trajectory coordinates, Na density values, density integral, and density planning approval ID are byte/identity identical. No wall-time speedup is claimed from this architecture-only migration because text trajectory parsing and the same qualified scientific owner dominate the bounded smoke; the qualified reduction is elimination of duplicate dependency authority and guaranteed one-execution reuse.

GFX3D-5 - universal rendering and interaction semantics

Implemented authority (0.20.150a0)

GFX3D-5 is implemented. Built-in layer adapters now materialize `renderer-neutral PointSet3D`, `SegmentSet3D`, `PolylineSet3D`, `TriangleMesh3D`, and `CellWireframe3D` primitives from already-prepared scientific products. `render_graphics3d_plotly()` contains no built-in framework/connectivity/trajectory/density dispatch; it understands only the common primitive vocabulary and therefore accepts newly registered layer types without changing the common result schema. The temporary `source_scene` renderer reference has been removed from prepared built-in layer products.

Named layers are Plotly legend groups with group-click visibility semantics. Declaration order remains canonical while the render-only integer `render_priority` may change backend draw order without changing scientific identity. `initially_visible` and scene-level `visible_layers` determine interactive starting state only. Camera projection, named

[100]/[110]/[111]/isometric presets, explicit camera eyes, background/axis controls, and width/height are reproducible view records.

Periodic display replication is one scene-wide view transformation. `periodic_images` accepts a canonical reference image, rectangular $N \times M \times K$ counts, or explicit lattice-image shifts. The same lattice shift is applied to every prepared primitive, so framework, connectivity, trajectories, densities, and future layers cannot drift into different periodic display gauges. Display replication does not duplicate scientific occupancy, density normalization, trajectory weights, or scientific evidence.

The renderer measures generic trace, point, segment, face, and estimated geometry-byte payload before Plotly construction and checks the selected browser profile. Budget failure is explicit; requested layers are never silently dropped. Per-layer payload contribution, primitive count, display-replication count, render priority, view state, and browser budget are render evidence only.

The supplied Na-LTA trajectory remains scientifically invariant against 0.20.149a0: mean-framework coordinates, Na trajectory coordinates, Na density values/integral, and density-planning authority are exactly identical. A stride-500 21-frame renderer-neutral scene with $2 \times 1 \times 1$ periodic display produces 20 Plotly traces, 149,896 mesh faces, and about 5.46 MB of estimated geometry payload; only the requested Na-density layer is initially visible in the qualification view.

Acceptance - satisfied

- no generic renderer code contains science-domain dispatch for built-in layer types;
- adding a mock fifth layer through the registry requires no common result-schema change;
- visibility/camera/periodic-display changes do not alter scientific identities;
- render priority changes backend ordering only;
- browser budget failures remain explicit rather than silently dropping requested layers;
- built-in prepared layer products no longer carry a renderer-only `source_scene` reference.

Post-foundation scientific visualization gates

After GFX3D-1 through GFX3D-5, future scientific products plug into the stable layer architecture.

A recommended sequence is:

GFX3D-RING1	ring visualization adapter
GFX3D-CAGE1	cage/natural-tiling visualization adapter
GFX3D-SITE1	discovered-site visualization adapter
GFX3D-SITE2	site assignments / basin classification display
GFX3D-KIN1	transition-path display
GFX3D-KIN2	site-kinetics / Markov-network display

These gates are placeholders, not approved scientific specifications. Their scientific definitions must come from the corresponding analysis architecture.

GFX3D-1 detailed object model

The first gate should prefer small immutable records over one universal mutable object.

A recommended starting decomposition is:

```
GraphicsScene3DRequest
  scene options
  ordered layer requests
  view request
  resource request
  output request
```

```
GraphicsLayer3DRequest
  name
  type
  selection
  analysis options
  render options
  initially_visible
```

GraphicsSelection
typed selector fields

GraphicsDependencyRequest
provider type
canonical scientific key
required/optional role

GraphicsSceneContext
source
resolved mappings
shared gauge
resource authority
cache registry
progress

PreparedGraphicsLayer3D
name/type
scientific identity
prepared product refs
primitives or primitive-preparation data
provenance

PreparedGraphicsScene3D
ordered layers
display gauge
scientific manifest/evidence

Graphics3DRenderResult
renderer artifact
layer-keyed results
render evidence

The exact Python inheritance model is less important than keeping these ownership roles distinct.

Suggested module boundaries

A target source layout may be:

```
mdstats/  
  graphics3d/  
    __init__.py  
    contracts.py  
    selection.py  
    identity.py  
    manifest.py  
    registry.py  
    dependencies.py  
    context.py  
    scene.py  
    primitives.py  
    presets.py  
    renderers/  
      __init__.py  
      plotly.py  
  layers/  
    __init__.py  
    framework.py  
    connectivity.py  
    trajectory.py  
    density.py
```

```
cli/  
graphics3d.py
```

```
tools/  
mdstats-3d.py
```

Alternative package naming such as `mdstats.plotting.scene3d` is acceptable if it preserves the dependency rules. The existing `mdstats.plotting` package may continue to own scientific plot adapters during migration.

The universal contracts should not be placed inside `framework_dynamics.py`; doing so would preserve the wrong ownership direction.

Implementation source map for 0.20.145a0

The migration begins from these concrete current sources:

Source	Current responsibility	GFX3D target
<code>examples/ plot_lta_mixed_alkali_density.py</code>	LTA-specific all-on workflow and CLI	compatibility preset + thin example; generic workflow moves to CLI/package
<code>mdstats/ plotting/framework_dynamics.py</code>	registration, mean framework, trajectory, atomic mean graph, density scene, composite render wrapper	scientific providers/adapters plus legacy compatibility facade
<code>mdstats/plotting/graph_3d.py</code>	Plotly graph rendering and HTML result	Plotly renderer backend and reusable graph primitive rendering
<code>mdstats/ plotting/periodic_graph.py</code>	periodic graph display materialization	shared periodic display service/primitive adapter
<code>mdstats/plotting/ framework_topology_graph.py</code>	framework topology view	framework layer scientific/primitive adapter
<code>mdstats/plotting/ atomic_connectivity_graph.py</code>	atomic connectivity view	connectivity layer scientific/primitive adapter
<code>mdstats/ plotting/atomic_density.py</code>	atomic density science/render options	density scientific provider + density layer adapter
density PAR-DENS modules	resource-aware density planning/execution	retained as density provider infrastructure, coordinated by scene context

This source map is descriptive, not a mandate to physically move every module immediately.

Deliberate limitations

The first GFX3D architecture does **not** require:

- a public third-party plugin API;
- a graphical GUI editor;
- live interactive recomputation of scientific products in the browser;
- WebGPU or custom JavaScript rendering;
- arbitrary free-form selection expressions;
- static publication rendering in the first gate;
- remote/distributed rendering;
- animation of every layer type;
- scientific definitions for rings, cages, sites, or kinetics that do not yet exist;
- replacement of the qualified density resource architecture;

- one universal styling schema that erases layer-specific graphical needs.

The architecture enables these extensions without requiring them now.

Acceptance criteria for the universal architecture

The GFX3D architecture is considered successfully established when all of the following are true:

1. The current hybrid LTA plot is no longer the only natural entry point for combined 3-D visualization.
2. A scene is represented as an ordered collection of named independent layer instances.
3. Framework topology, atomic connectivity, trajectory, and density are separate initial layer classes.
4. All non-empty combinations of those initial classes are supported and tested.
5. Multiple instances of one layer type are supported.
6. The universal selection architecture is shared across layer families while preserving layer-specific validation.
7. One scene context owns source/gauge/resource/cache authority.
8. Scientific dependencies are declared, deduplicated, and reusable.
9. Renderers cannot initiate undeclared scientific analysis.
10. Scientific, render, and execution identities are explicitly separated.
11. Periodic display replication and camera state are render/view operations rather than scientific duplication.
12. A canonical scene manifest can reproduce the scientific request and view configuration.
13. Layer types are registry-driven rather than hard-coded into one common dispatcher.
14. Generic render results are keyed by layer name rather than fixed fields for every science domain.
15. The source-checkout CLI is a thin launcher around packaged implementation.
16. TOML, presets, CLI shorthand, and Python construction converge on the same canonical request.
17. The existing hybrid LTA workflow is available as a compatibility preset/configuration rather than special code.
18. The density/framework manual retains density science and marks generic composition as GFX3D-owned.
19. A separate formal CLI specification and concise user guide accompany the implemented CLI.
20. Ring/cage/site/kinetics visualization can be added through new registered layers without changing the universal scene schema.

One-sentence architectural rule

Scientific subsystems produce immutable spatial evidence; GFX3D resolves and shares their dependencies, composes any requested named layers in one coordinate gauge, and hands renderer-neutral prepared content to a renderer whose choices never redefine the science.

Recommended next implementation

GFX3D-1 through GFX3D-5 are complete through `mdstats 0.20.150a0`. The universal scene/dependency/rendering foundation is therefore closed. The next visualization work should begin with the first scientific extension gate, **GFX3D-RING1 - ring visualization adapter**, after confirming the ring scientific authority to expose from the framework/ring subsystem. Cage, site, assignment, transition-path, and Markov-network layers can then follow without another common scene redesign.

Revision record

2026-08-11 - GFX3D-5 implementation (0.20.150a0)

Removed the remaining legacy composite-renderer coupling from `render_graphics3d_plotly()`. Built-in adapters now emit renderer-neutral primitive records directly from prepared framework, connectivity, trajectory, and density products. Added generic primitive-only Plotly realization, layer legend groups, visibility overrides, integer render priority, scene-wide periodic image replication, reproducible camera/view presets, and generic browser payload/budget evidence. The renderer has no built-in layer-family dispatch and a mock fifth registered layer renders without a result-schema change.

Focused qualification covers all GFX3D-1 through GFX3D-5 contracts plus the legacy framework/graph renderers: 83 passed. A real 21-frame Na-LTA stride-500 qualification verifies zero `source_scene` renderer references in prepared layer products, one underlying source preparation, exact scientific-product equality to `0.20.149a0`, `2x1x1` shared periodic replication, camera/visibility semantics, and explicit browser payload accounting.

2026-08-11 - GFX3D-4 implementation (0.20.149a0)

Replaced the raw CLI's monolithic `framework_dynamics_scene` layer dependency with product-level `framework_topology_product`, `atomic_connectivity_product`, `atomic_trajectory_product`, and `atomic_density_product` keys. Added `Graphics3DDependencySource` / `GraphicsScientificProduct`, the `LTAGraphics3DDependencySource` raw-source authority, single-flight concurrent cache resolution, deterministic dependency-plan collation, dependency timing/cache evidence, explicit in-memory/no-durable-cache policy, and source-level qualified batching under the existing framework-dynamics resource scheduler. Legacy prepared `FrameworkDynamicsScene` inputs remain supported through the historical adapter path.

Focused qualification covers product-key planning, duplicate density dependency deduplication, eight-way concurrent single-flight execution, cache-state-neutral identity, LTA source one-preparation batching, GFX3D-1/2/3 compatibility, and the existing framework/Plotly renderer. A real Na-LTA stride-500 four-family preparation resolves four product dependencies from one source preparation; a three-family framework + Na trajectory + Na density run omits connectivity and generates the expected self-contained HTML. Mean-framework coordinates, Na trajectory coordinates, Na density values/integral, and density planning approval identity are exactly unchanged from 0.20.148a0.

2026-08-11 - GFX3D-3 implementation (0.20.148a0)

Promoted the prototype workflow into the universal packaged `mdstats-3d` CLI and source-checkout tools/`mdstats-3d.py` launcher. Added strict TOML configuration, source-aware preset expansion, layer shorthand, deterministic configuration precedence, source/input resolution, canonical manifest-only mode, protected HTML/manifest outputs, concise fail-closed errors, and the `lta-mixed-alkali-density` compatibility preset. The historical example path now delegates to the same command authority while preserving its input helpers and default output behavior.

Focused qualification covers CLI/TOML compilation, shorthand, source-aware preset expansion, precedence, overwrite protection, manifest-only source resolution, all GFX3D-1/2 contracts, the existing framework/Plotly stack, and historical example input compatibility. Result: 83 passed. A real Na-LTA source-tree smoke on the authenticated 10,001-frame dump used stride 500 (21 frames), compiled framework + trajectory:Na + density:Na, wrote the canonical manifest, and produced a self-contained 6.5 MiB HTML under a bounded density configuration. GFX3D-4 is the next gate.

2026-08-11 - GFX3D-2 implementation (0.20.147a0)

Implemented the four initial independent registered layer adapters: `FrameworkTopologyLayer`, `AtomicConnectivityLayer`, `AtomicTrajectoryLayer`, and `AtomicDensityLayer`. Added `prepare_graphics3d_scene()` over a shared `GraphicsSceneContext` dependency and a common `render_graphics3d_plotly()` composer with generic layer-owned trace results. The migration path consumes already-prepared `FrameworkDynamicsScene` products and does not recompute or redefine current framework, connectivity, trajectory, or density science.

Focused qualification covers all 15 non-empty initial layer combinations, duplicate trajectory/density instances, pair-filtered connectivity, fail-closed absent-density selection, GFX3D-1 contract regressions, and the existing framework/Plotly path. Result: 62 passed. GFX3D-3 is the next gate.

2026-08-11 - GFX3D-1 implementation (0.20.146a0)

Implemented the universal contract foundation without changing scientific plotting algorithms. The new `mdstats.graphics3d` package provides immutable scene/layer/selection/dependency records, deterministic canonical identities and manifests, a shared scene-context cache foundation, deterministic internal layer registry, renderer-neutral primitive contracts, generic layer-keyed render-result contracts, and compatibility adapters for the current `FrameworkDynamicsScene` and `FrameworkDynamicsRenderResult`.

Focused qualification covers deterministic serialization, unique layer names, scientific/render/execution identity separation, dependency deduplication, registry determinism, selection fail-closed behavior, immutable primitive geometry, and the current framework-dynamics/Plotly compatibility path. The next gate is GFX3D-2.

2026-08-11 - Initial universal GFX3D architecture

Created the canonical architecture for promoting the LTA hybrid density example into a universal configurable three-dimensional graphics subsystem.

The initial standard establishes:

- named composable layers;
- separate framework/connectivity/trajectory/density layer families;
- shared scene context and dependency DAG;
- universal selection architecture;
- scientific/render/execution identity separation;
- canonical manifests;
- registry-driven extensibility;
- renderer-neutral primitives;
- Plotly as the first renderer rather than the scene model;
- preset-based compatibility for the current LTA workflow;
- staged GFX3D-1 through GFX3D-5 implementation;
- future ring/cage/site/kinetics integration without universal-scene redesign.

2026-08-11 - GFX3D-HARDEN1 implementation (0.20.156a0)

After the GFX3D-1 through GFX3D-5 foundation was closed, a robustness audit of the four existing layer families identified several implementation seams that were safe scientifically but undesirable before ring/cage/site growth. 0.20.156a0 closes those seams without introducing a new scientific layer type.

The raw-source topology sidecar is now an authenticated `mdstats.graphics3d.topology-cache.v2` record. Cache reuse requires exact agreement in trajectory topology identity (atom identities, frame IDs, cells, fractional coordinates, PBC and frame semantics), the resolved framework-connectivity definition, and framework mapping. A mismatched, malformed, or legacy unauthenticated sidecar is never silently reused.

`GraphicsScientificProduct` no longer carries a `FrameworkDynamicsScene` compatibility object. It carries only the requested scientific value plus renderer-neutral display-cell/frame/provenance context; density uses `GraphicsDensityProduct` to expose atomic/framework fields and atom identities explicitly. This makes the dependency boundary consistent with the GFX3D-4/5 rule that a product must not smuggle the old composite scene back into universal layer preparation.

Cell wireframes are now scene/view primitives rather than framework-layer primitives. Consequently a density-only or trajectory-only scene can display the same reference cell. Camera and periodic-replication declarations are strict: non-finite or zero camera vectors and non-integral periodic counts/origins fail closed. Browser payload is estimated and checked before display-only periodic copies are materialized, then checked again after realization. Trajectory hover metadata is restored on renderer-neutral polylines.

Framework preprocessing receives two execution-only hardening paths. Uniform catalogs bypass redundant partitioned preparation. For the LTA GFX3D provider, partitioned catalogs prepare only the dominant category needed to reproduce the current framework/connectivity products; the complete catalog remains scientific evidence, while non-dominant category render objects are omitted because no current universal layer consumes them. The public/legacy framework-dynamics API retains full-category materialization by default.

Long static-cell trajectories use a bulk projected-geometry path: all retained atomic-path segment MIC vectors are evaluated in one exact triclinic batch, graph gauges are propagated across the frame axis, and only canonical graph-view metadata is retained. Variable-cell and periodic-multiedge cases retain the frame-local reference path. First-frame shift/lift equivalence is checked before the bulk result is accepted. On the supplied 10,001-frame Na-LTA trajectory, fixed-topology framework preparation completed in about 11.23 s; the projected framework-registration stage was about 0.5 s. A 1,001-frame fixed-topology preparation measured about 1.17 s after the bulk path versus about 13.17 s immediately before it.

This hardening gate does not change density normalization, HDR semantics, framework mapping, connectivity cutoffs, trajectory registration semantics, layer scientific identities, or the renderer-neutral primitive vocabulary. GFX3D-RING1 remains the next scientific-extension gate.

2026-08-12 - GFX3D-HARDEN2 implementation (0.20.157a0)

A preparation-performance and failure-path audit after HARDEN1 found that the product-level dependency DAG still converged on one qualified LTA science owner, but the owner cached only successful preparation. Concurrent product requests could therefore serialize behind the same lock and, after a failure, retry the entire expensive preparation one after another. HARDEN2 makes the source owner single-flight for both success and failure: the first attempt is authoritative, a failure is

latched with its original exception, and every dependent product receives the same causal failure. The CLI now renders the complete exception chain so a density/topology/science error is not hidden by whichever product key first triggered source preparation.

The LAMMPS custom-dump adapter now applies positive start/stop/stride selection during streaming scan. Unselected atom tables are consumed but not tokenized/materialized, while every source frame is still counted for exact provenance. Negative slice bounds deliberately retain full materialization to preserve Python negative-index semantics. On the supplied 10,001-frame, roughly 183 MiB Na-LTA dump, isolated stride=500 input retained exactly 21 frames and source_frame_count=10001; measured read time fell from about 9.17 s to 3.03 s and peak RSS from about 1.63 GiB to 253 MiB in the qualification environment.

The LTA source now shares one AtomicConnectivityGeometryCache between the framework-only hysteretic connectivity pass used by topology construction and the later full atomic-connectivity pass. This reuses identical pair geometry without changing pair definitions, hysteresis, state construction, or topology identity. Preparation also exposes explicit calibration/connectivity/topology/ scene progress stages and warns on implausible T-O calibration or pathological topology fragmentation, which makes a wrong numeric type map or damaged framework visible rather than presenting only as unexplained preparation cost.

Atomic-density Phase B now retains an immutable compact AtomicDensityResolvedPlan containing the selected atom identity, registration signature, options, and resolved numerical policy. Realization consumes that approved policy instead of resolving adaptive grid/bandwidth numerics a second time. Coordinate samples are intentionally not retained in the plan so this optimization does not create a second trajectory-sized memory resident set or bypass the established resource planner.

The universal density renderer now recognizes packed sparse scalar fields and uses the qualified sparse mesh/node-cloud machinery. Per-shell face counts are visual targets under the scene-level browser controller rather than an independent hard cap that would force expensive simplification before the aggregate GFX3D browser budget is evaluated. This repairs the previous PeriodicPackedBlockScalarField3D .values failure without redefining density normalization, HDR thresholds, or sparse scientific values.

The historical lta-mixed-alkali-density preset remains unchanged: all present supported species still receive density layers. This can be intrinsically expensive for nearly static framework atoms because adaptive smearing may demand very fine logical grids. Mobile-ion-only scenes should explicitly request, for example, density:Na; changing the compatibility preset silently would be a semantic regression. A proposed one-pass dominant_only framework shortcut was also rejected because it changed qualified coordinates at approximately $1e-16$; the exact legacy-equivalence contract remains authoritative.

2026-08-12 - GFX3D-HARDEN3 implementation (0.20.158a0)

HARDEN3 closes the long-trajectory atomic-connectivity and atomic-mean-graph failures exposed after HARDEN2. The general minimum-image engine previously computed a correct vector in a Minkowski-reduced basis, then attempted to recover the associated integer image shift by multiplying through the inverse of the original cell and rounding. That post-hoc reconstruction is not numerically stable for ill-conditioned representations. The revised kernel tracks the reduced-basis integer wrap/search coefficients directly and maps them back through the exact unimodular Minkowski operation. Consistency is certified in the numerically stable reduced basis; the original-basis integer label remains an exact algebraic consequence of the transform.

Atomic connectivity now avoids several Python-scaling traps. Canonical graph construction is array-oriented, transition differences are linear merges over canonical sorted endpoints, and the raw-state construction cache is capped at 512 entries instead of growing with the number of distinct thermal graph states. For fixed fully periodic cells, the cell-list search basis and metric stencil are cached by immutable cell/PBC/cutoff identity. LTA-style heterospecies registries whose cutoff pairs share oxygen are evaluated by one exact star request followed by pair-specific inner/outer filtering.

HARDEN2's trajectory-wide cross-pass geometry cache is removed from the LTA provider. If the scene requests both atomic connectivity and framework topology, the broader hysteretic graph is computed once and the framework pair/scope subset is projected from its canonical states. The projection is accepted only when target formation/breaking cutoffs are exactly identical to the corresponding source pairs and both definitions use formation-cutoff initialization. Real-trajectory qualification shows identical framework state digests, frame-state IDs, and transitions versus an independent direct pass. This provides bounded memory and removes a complete neighbor-geometry traversal.

Atomic mean-graph periodic averaging now uses a certified fast path. A circular-start Fréchet solution may bypass the historical weighted-medoid / multi-start search only when every weighted sample lies inside a conservative strong-convexity ball whose radius is one quarter of the shortest nonzero periodic translation. This guarantees a unique local/global torus mean in the accepted regime. Any distribution that fails the certificate retains the original exact multi-start implementation.

Threading was evaluated but is not enabled blindly for the hysteretic connectivity fold. The existing frame-threaded candidate implementation loses sequential reuse and materializes candidate geometry before the authoritative stateful fold; on the supplied fixed-cell LTA workload it is slower than the optimized serial cell-list/star path. HARDEN3 therefore improves asymptotic and constant-factor execution without making CPU utilization a proxy for performance. A future parallel connectivity design, if needed, should use a bounded producer/fold pipeline or compiled geometry kernel rather than the existing all-candidates thread path.

Qualification includes ill-conditioned MIC image-shift regression against ASE, certified-versus-authoritative periodic means, exact star-batch versus pair-by-pair neighbor enumeration, exact framework projection, fixed-cell cell-list plan reuse, and the existing GFX3D/framework/neighbor/topology suite. A 400-frame cold full-connectivity benchmark on the supplied Na-LTA trajectory changed from about 6.59 s in 0.20.157a0 to about 1.59 s with the same connectivity identity SHA-256. A full 10,001-frame framework + connectivity + Na-trajectory run completed successfully; atomic connectivity resolved in about 41.5 s, framework projection in about 1.3 s, and the former atomic_mean_graph MIC failure did not recur.

2026-08-12 - GFX3D-HARDEN4 implementation (0.20.159a0)

HARDEN4 closes an execution-contract gap exposed by the adaptive GFX3D density preset. PAR-DENS correctly admitted independent fields and dynamically rebalanced CPU tokens, but the LD8-S3 direct sparse tile executor used the lease only indirectly for FFT worker selection. Direct tiles therefore remained effectively one-core kernels even when the scheduler had granted many workers. On large adaptive grids this presented as a long density_realization [0/4] interval with little visible host utilization.

The direct executor now performs periodic target construction and packed-target lookup concurrently over contiguous source-row slices of the *same already-approved pair chunk*. This boundary is important: parallelism cannot create several full pair chunks because Phase-B memory admission prices one bounded chunk. A shared mapped-index array is filled at the exact canonical flattened offsets, after which the historical stable grouped reduction runs once over the complete pair sequence. Thus worker count changes execution time only; it cannot alter pair ordering or floating-point reduction order. The transient contract is conservatively raised from 96 to 112 bytes per pair to cover the shared mapped-index buffer without multiplying memory by worker count.

The worker pool is field-local, lazily created, and sized to the task's preferred lease ceiling. Every chunk queries the live cooperative lease, so a field that begins with one worker while siblings are active can expand after those siblings complete. This makes PAR-DENS3's water-filling allocation effective for the direct sparse regime that GFX3D adaptive framework/mobile-ion densities actually use.

HARDEN4 also makes scheduler state observable. The realization scheduler receives the scene progress port and reports task admission, backend, worker allocation, declared memory peak, task completion, and maximum workers. The hybrid executor reports direct-pair progress approximately every two seconds and emits before long FFT calls. Direct sparse progress explicitly identifies itself as CPU execution; GPU activity is neither required nor expected unless the frozen Phase-B plan contains FFT tiles and the optional GPU policy admits them.

Qualification requires bitwise worker-count invariance. A forced-direct 8,421,376-pair regression produces np.array_equal packed values for one-worker and four-worker realization. In the 3-CPU-token packaging environment, the supplied Na-LTA stride=500 Na-only density stage changed from about 4.8 s to 3.8 s, and the four-density preset changed from about 20.4 s to 16.8 s. The O field demonstrably grew from one to three live workers after its siblings completed. These factors are environment-specific and are evidence rather than scientific constants.